

Lecture 6 : Circular Dependency, Bean Scope & Bean Initialization in Spring

1. Quick Recap: DI, IoC and Beans

Before understanding circular dependency, bean scopes, and initialization, we should already be clear about three basic Spring concepts:

Concept	Meaning
DI	Dependency Injection. Spring provides the required dependency to a class instead of the class creating it manually.
IoC	Inversion of Control. The control of object creation and dependency management is given to the Spring container.
Bean	An object created, managed, and maintained by the Spring IoC container.

In Spring, classes annotated with annotations like `@Component`, `@Service`, `@Repository`, `@Controller`, or methods annotated with `@Bean` can become Spring-managed beans.

2. `@Configuration` Is Also a Spring Component

In Spring, `@Configuration` is used to define configuration classes.

Example:

```
@Configuration
public class AppConfig {
}
```

A configuration class can contain `@Bean` methods:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

An important point is that `@Configuration` itself internally contains `@Component`.

That means a class annotated with `@Configuration` is also detected by component scanning and registered as a Spring bean.

So we can understand it like this:

```
@Configuration
    ↓
@Component
    ↓
Spring can detect and manage this class
```

Part 1: Why Bean Creation Order Matters

3. Can Spring Create Beans in Any Random Order?

No.

Spring does not create beans blindly in random order. It creates beans based on their dependencies.

Suppose we have this dependency chain:

```
OrderController depends on OrderService
OrderService depends on PaymentService
PaymentService depends on PaymentGateway
```

Before Spring can create `OrderService`, it needs `PaymentService`.

Before it can create `PaymentService`, it needs `PaymentGateway`.

So the creation order becomes:

```
PaymentGateway
  ↓
PaymentService
  ↓
OrderService
  ↓
OrderController
```

Spring tries to create the dependency first, and then the class that depends on it.

4. Why This Matters for Circular Dependency

Now imagine this situation:

```
A needs B
B needs A
```

Spring starts creating **A**.

But to create **A**, it needs **B**.

Then Spring starts creating **B**.

But to create **B**, it needs **A**.

So the container gets stuck in a loop:

```
To create A, I need B.
To create B, I need A.
To create A, I need B.
...
```

This is the circular dependency problem.

Spring creates beans based on dependency order. But when the dependency order becomes circular, Spring cannot find a clear starting point.

Part 2: Circular Dependency

5. What Is Circular Dependency?

Circular dependency means two or more classes depend on indirectly.

Example:

```
OrderService depends on PaymentService
PaymentService depends on OrderService
```

Example Code

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {

    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Here, Spring cannot decide which object should be created first.

To create `OrderService`, Spring needs `PaymentService`.

To create `PaymentService`, Spring needs `OrderService`.

This creates a deadlock-like situation during bean creation.

6. Circular Dependency Is Not Only a Spr

Circular dependency is not only a Spring-specific issue. It can happen in plain Java as well.

Simple Java Example

```
public class A {  
  
    private B b;  
  
    public A() {  
        this.b = new B();  
    }  
}
```

```
public class B {  
  
    private A a;  
  
    public B() {  
        this.a = new A();  
    }  
}
```

Now suppose we write:

```
public class Main {  
    public static void main(String[] args) {  
        A a = new A();  
    }  
}
```

The flow becomes:

```
new A()  
→ A constructor runs  
→ new B()  
→ B constructor runs  
→ new A()  
→ A constructor runs
```

```
→ new B()  
    → new A()  
        → ...
```

This never ends.

Eventually, Java throws:

```
StackOverflowError
```

This happens because constructor calls keep getting stacked again and again.

So circular dependency is a design problem. Spring simply exposes this problem during bean creation.

Part 3: Constructor-Based Circular Dependency

7. Why Constructor Injection Fails in Circular Dependency

Constructor injection has one strict rule:

| An object cannot be created until all constructor arguments are available.

For example:

```
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

To create `OrderService`, Spring must already have a `PaymentService` object.

But if `PaymentService` also needs `OrderService` in its constructor, Spring cannot create either of them first.

The loop becomes:

```
Create OrderService → needs PaymentService
Create PaymentService → needs OrderService
Create OrderService → needs PaymentService
...
```

There is no starting point.

That is why constructor-based circular dependency fails.

Spring detects this situation during startup and throws an exception such as:

```
BeanCurrentlyInCreationException
```

The application fails to start.

8. Why Constructor Injection Is Still Recommended

Even though constructor injection fails in circular dependency, constructor injection is still the preferred style in most cases.

Why?

Because it makes dependencies clear and mandatory.

If a class cannot work without a dependency, constructor injection clearly communicates that.

Example:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

This tells us:

```
OrderService cannot exist without PaymentService.
```

So constructor injection is not the problem. The real problem design.

Part 4: Setter and Field Injection in Circular Dependency

9. Circular Dependency Using Setter Injection

Now let us rewrite the same example using setter injection.

```
@Service
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {

    private OrderService orderService;

    @Autowired
    public void setOrderService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

In setter injection, object creation and dependency injection are separate steps. Spring can conceptually do this:


```
Step 1: Create empty OrderService object
Step 2: Create empty PaymentService object
Step 3: Inject PaymentService into OrderService
Step 4: Inject OrderService into PaymentService
```

This is different from constructor injection.

In constructor injection:

```
Dependency must exist before object creation.
```

In setter injection:

```
Object can be created first.
Dependency can be injected later.
```

Because of this difference, setter injection may sometimes allow Spring to resolve circular dependency.

10. Circular Dependency Using Field Injection

The same thing can happen with field injection.

```
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

```
@Service
public class PaymentService {

    @Autowired
    private OrderService orderService;
}
```

Here also, Spring can create the object first and inject the de
But this does not mean circular dependency is a good practi
It only means Spring may be able to handle some circular de
object creation and dependency injection are separated.

11. Why Setter/Field Circular Dependenc

In setter or field injection, the constructor does not require the dependency immediately.

Example:

```
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    public OrderService() {
        System.out.println("OrderService object created");
    }
}
```

Here, Spring can create the `OrderService` object without immediately needing `PaymentService`.

Similarly:

```
@Service
public class PaymentService {

    @Autowired
    private OrderService orderService;

    public PaymentService() {
        System.out.println("PaymentService object created");
    }
}
```

Spring gets some flexibility.

Conceptually, Spring can say:

```
I have created OrderService object,  
but it is not fully initialized yet.  
  
Still, I can keep an early reference to it.  
  
Now I create PaymentService.  
PaymentService needs OrderService.  
  
I can give it the early reference of OrderService.  
  
Then I come back and inject PaymentService into OrderService.
```

This is the basic idea behind Spring's early reference mechanism.

Part 5: Spring's Early Reference Concept

12. Normal Dependency Flow

In a normal dependency situation:

```
A needs B
```

Spring creates **B** first.

Then it injects **B** into **A**.

```
Create B  
Inject dependencies into B  
Initialize B  
B is ready  
  
Create A  
Inject B into A  
Initialize A  
A is ready
```

The dependent bean receives a fully created and initialized dependency.

13. Circular Dependency Flow with Early

In a circular dependency:

```
A needs B  
B needs A
```

Spring may try to create one object first and expose an early reference to it.

Conceptually:

```
A a = new A(); // A exists, but dependencies are not injected yet  
  
B b = new B(); // B exists  
  
b.setA(a);      // B receives early reference of A  
a.setB(b);      // A receives B
```

An early reference means:

Spring exposes a reference of a bean before the bean is fully initialized, so another bean can temporarily use that reference during circular dependency resolution.

This can help Spring resolve some setter/field-based circular dependencies.

14. Important Spring Boot Point

Spring Framework has internal mechanisms to resolve some setter or field-based circular dependencies.

However, Spring Boot discourages circular references.

From Spring Boot 2.6 onward, circular references are disabled by default.

The related property is:

```
spring.main.allow-circular-references=false
```

Its default value is:

```
false
```

You can technically enable it:

```
spring.main.allow-circular-references=true
```

But this should not be treated as the solution.

The better solution is to improve the design.

Part 6: Why Circular Dependency Is a Design Smell

15. Circular Dependency Usually Indicates Mixed Responsibilities

Circular dependency is usually not only a technical issue.

It often shows that responsibilities are not clearly separated.

Ask these questions:

```
Why does A need B?  
Why does B also need A?  
Are both classes doing too much?  
Can we move common logic to a third class?  
Can one dependency direction be removed?
```

Example:

```
OrderService depends on PaymentService  
PaymentService depends on OrderService
```

This may indicate that order-related logic and payment-related logic are mixed.

A better design may be:

```
OrderService → PaymentService
```

or

```
OrderService → PaymentService
```

```
PaymentService → PaymentStatusRepository
```

or

```
OrderService → PaymentProcessor
```

```
PaymentService → PaymentProcessor
```

Instead of both depending on each other.

16. Best Way to Solve Circular Dependency

The best solution is refactoring.

Possible ways:

Problem	Better Design
Two services call each other	Move shared logic to a third service
One service has too many responsibilities	Split responsibilities
Both classes depend on each other's internal behavior	Introduce an interface or event-based communication
Service is doing orchestration and business logic together	Create a separate coordinator/orchestrator class

The first principle is simple:

└ Avoid circular dependency by designing clear responsibility boundaries.

Part 7: Bean Scope

17. What Is Bean Scope?

Bean scope decides:

How many objects Spring will create for a bean
and how long those objects will live.

The important question is:

How many objects will Spring create for one bean definition?

Spring provides different scopes for different use cases.

The two most important core scopes are:

singleton
prototype

In web applications, Spring also provides:

request
session
application
websocket

Part 8: Singleton Scope

18. What Is Singleton Scope?

Singleton scope means:

Spring creates exactly one object for a bean definition inside the Spring container, stores it, and returns the same object whenever needed.

Singleton is the default scope in Spring.

So this:

```
@Component
public class PaymentService {
}
```

is the same as:

```
@Component
@Scope("singleton")
public class PaymentService {
}
```

Usually, we do not need to write `@Scope("singleton")` explicitly.

19. Singleton Scope Demo

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);

PaymentService p1 = context.getBean(PaymentService.class);
PaymentService p2 = context.getBean(PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
true
```

Both references point to the same Spring-managed singleton bean.

20. Important Clarification About Singleton

Spring singleton does not mean:

```
Only one object of this class can ever exist in the JVM.
```


We can still manually create an object:

```
PaymentService p = new PaymentService();
```

But this object is not managed by Spring.

Spring singleton means:

Inside this Spring container, for this bean definition, Spring keeps one shared object.

21. Singleton Is Per Bean Definition, Not Per Class

Consider this example:

```
@Bean
public User createUser() {
    return new User();
}

@Bean
public User createUser2() {
    return new User();
}
```

Spring creates two different bean definitions:

```
createUser
createUser2
```

Both are singleton by default.

So Spring creates:

```
createUser → one User object
createUser2 → one User object
```

Total:

```
2 bean definitions
2 singleton objects
```

Therefore:

In Spring, singleton means one object per bean name or b
the Spring container.

It does not mean one object per class.

This is different from the traditional Singleton Design Pattern.

Part 9: Prototype Scope

22. What Is Prototype Scope?

Prototype scope means:

Spring creates a new object every time that bean is requested from the
container.

Example:

```
@Component
@Scope("prototype")
public class OrderRequest {

    public OrderRequest() {
        System.out.println("OrderRequest object created");
    }
}
```

Now if we do:

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);

OrderRequest r1 = context.getBean(OrderRequest.class);
OrderRequest r2 = context.getBean(OrderRequest.class);
```

```
System.out.println(r1 == r2);
```

Output:

```
false
```

Why?

Because each request gives a new object.

23. Prototype Feels Closer to **new**

Prototype scope feels closer to manual object creation using **new**.

Every time we ask Spring for the bean, it gives a fresh object.

```
context.getBean(OrderRequest.class) → new object  
context.getBean(OrderRequest.class) → new object  
context.getBean(OrderRequest.class) → new object
```

But the object is still created by Spring, not manually by us.

24. When to Use Singleton and Prototype

A simple rule:

Scope	Best For
Singleton	Stateless beans that provide behavior
Prototype	Stateful beans that store changing data

Singleton Example

Services are usually singleton:

```
@Service  
public class PaymentService {
```

```
}
```

Why?

Because services usually contain behavior, not user-specific

Example:

```
processPayment()  
validateOrder()  
sendEmail()  
calculateDiscount()
```

These methods can be reused safely.

Prototype Example

Prototype can be used when each request needs a fresh object with its own state.

Example:

```
@Component  
@Scope("prototype")  
public class OrderRequest {  
}
```

Each time we need a new `OrderRequest`, Spring can create a fresh object.

25. Prototype Bean Injected Into Singleton Bean

Important question:

If we inject a prototype bean into a singleton bean, will the singleton get a new prototype object every time?

Answer:

No.

Why?

Because the singleton bean is created only once.

At the time of singleton creation, Spring injects one prototype

After that, the same singleton object keeps using the same in reference.

Example:

```
@Service
public class OrderService {

    private final OrderRequest orderRequest;

    public OrderService(OrderRequest orderRequest) {
        this.orderRequest = orderRequest;
    }
}
```

If `OrderService` is singleton and `OrderRequest` is prototype, then Spring injects one `OrderRequest` object when `OrderService` is created.

After that, the same `OrderRequest` reference remains inside the singleton.

So prototype does not automatically mean:

```
new object every time a singleton method runs
```

It means:

```
new object every time the prototype bean is requested from the container
```

Part 10: Web Scopes

26. Web-Specific Bean Scopes

Apart from `singleton` and `prototype`, Spring also provides web-specific scopes.

The important web scopes are:

Scope	One Object For
<code>request</code>	One HTTP request
<code>session</code>	One user session
<code>application</code>	One ServletContext / whole web application
<code>websocket</code>	One WebSocket session

These scopes are available only in a web-aware Spring `ApplicationContext`.

For beginner-level Spring MVC or Spring Boot Web understanding, the most important scopes are:

```
request
session
application
```

27. Request Scope

Request scope means:

| Spring creates one bean object for one HTTP request.

Example:

```
User sends request 1 → new request-scoped bean
User sends request 2 → new request-scoped bean
```

This is useful when data should live only during one HTTP request.

28. Session Scope

Session scope means:

| Spring creates one bean object for one user session.

Example:

```
User A session → one bean object
User B session → another bean object
```

This can be useful for user-specific session data.

29. Application Scope

Application scope means:

| Spring creates one bean object for the entire web application.

This bean is tied to the `ServletContext`.

It is shared across the web application.

Part 11: Bean Initialization

30. When Are Beans Actually Created?

Spring has to answer one important question:

```
When should I create this bean?
```

There are two main answers:

Initialization Type	Meaning
Eager initialization	Create the bean during application startup
Lazy initialization	Create the bean only when it is actually needed

By default:

```
Singleton beans are eagerly initialized.
Prototype beans are created lazily, when requested.
```

Lazy initialization can also be enabled using `@Lazy`.

Part 12: Eager Initialization

31. What Is Eager Initialization?

Eager initialization means:

| Spring creates the bean as soon as the application context

For singleton beans, this is the default behavior.

Example:

```
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService object created");
    }
}
```

When the application starts, Spring creates the singleton bean immediately.

32. Why Eager Initialization Is the Default

At first, eager initialization may look wasteful.

We may think:

Why create everything at startup if the user may not use it immediately?

But eager initialization has a big advantage.

It helps the application fail fast.

Suppose a bean has a configuration issue, missing dependency, or invalid setup.

With eager initialization, the application fails during startup.

That is useful because we find the problem early.

This is better than starting the application successfully and then failing only when the first user hits a feature.

Eager initialization gives:

```
Fast failure  
Early error detection  
Predictable startup validation
```

That is why Spring generally prefers eager singleton creation

Part 13: Lazy Initialization

33. What Is Lazy Initialization?

Lazy initialization means:

Do not create the bean during startup. Create it only when someone actually asks for it.

Example:

```
@Component  
@Lazy  
public class ReportService {  
  
    public ReportService() {  
        System.out.println("ReportService object created");  
    }  
  
    public void generateReport() {  
        System.out.println("Report generated");  
    }  
}
```

Here, Spring knows about `ReportService`, but it does not immediately create the object.

Spring only keeps the bean definition.

A bean definition contains information like:

```
class name
scope
dependencies
lifecycle details
bean name
```

The actual object is created only when someone asks for it:

```
ReportService reportService = context.getBean(ReportService.class);
reportService.generateReport();
```

Output:

```
Application started
ReportService object created
Report generated
```

34. Important Point: Lazy Bean Needed by Eager Bean

If a lazy bean is required by an eager singleton bean, then the lazy bean may still be created at startup.

Why?

Because Spring must satisfy the dependencies of the eager singleton.

Example:

```
@Component
public class UserService {

    private final ReportService reportService;

    public UserService(ReportService reportService) {
        this.reportService = reportService;
    }
}
```

If `UserService` is eager and needs `ReportService`, Spring has to while creating `UserService`.

So even if `ReportService` is marked lazy, it may still be created because another eager bean needs it.

Part 14: `@Lazy` on Injection Point

35. Lazy Dependency Injection Using Proxy

There is another interesting use of `@Lazy`.

Instead of making the whole bean lazy, we can mark the injection point as lazy.

Example:

```
@Component
public class UserService {

    private final EmailService emailService;

    public UserService(@Lazy EmailService emailService) {
        this.emailService = emailService;
        System.out.println("UserService created");
    }

    public void registerUser() {
        System.out.println("User registered");
        emailService.sendEmail();
    }
}
```

Here, Spring does not inject the real `EmailService` object immediately.

Instead, Spring injects a proxy object.

A proxy is like a placeholder.

Conceptually:

`UserService` does not receive the real `EmailService` immediately.

`UserService` receives an object that looks like `EmailService`.

```
When someone actually calls a method on it,  
Spring creates or fetches the real EmailService.
```

Startup flow:

```
UserService created  
Application started
```

Later, when this method runs:

```
emailService.sendEmail();
```

Spring resolves the real `EmailService`.

Output:

```
User registered  
EmailService created  
Email sent
```

36. Bean-Level `@Lazy` vs Injection-Point `@Lazy`

Usage	Meaning
<code>@Lazy</code> on class	Do not create this bean until it is requested
<code>@Lazy</code> on injection point	Inject a proxy and resolve the real dependency only when it is used

Example of bean-level lazy:

```
@Component  
@Lazy  
public class ReportService {  
}
```

Example of injection-point lazy:

```
public UserService(@Lazy EmailService emailService) {  
    this.emailService = emailService;  
}
```

Part 15: Global Lazy Initialization Boot

37. Enabling Global Lazy Initialization

Spring Boot can make the whole application lazy using this property:

```
spring.main.lazy-initialization=true
```

In YAML:

```
spring:  
  main:  
    lazy-initialization: true
```

This means beans are created only when needed instead of being created during startup.

This can reduce startup time.

But there is a trade-off.

Misconfiguration errors may be discovered later, only when the lazy bean is actually initialized.

38. Default Value

By default:

```
spring.main.lazy-initialization=false
```

So Spring Boot does not make all beans lazy by default.

39. Opting Out of Global Lazy Initialization

If global lazy initialization is enabled, we can still force a part to initialize eagerly.

Example:

```
@Component
@Lazy(false)
public class ImportantStartupBean {
}
```

This means:

| Even if global lazy initialization is enabled, create this bean eagerly.

Part 16: Common Lazy Initialization Scenarios

40. Eager Bean Depends on Lazy Bean

Eager Bean → Lazy Dependency

If an eager singleton directly needs a lazy dependency, Spring may still create the lazy dependency at startup to satisfy the eager bean.

Unless we use `@Lazy` at the injection point, where Spring can inject a proxy instead.

41. Lazy Bean Depends on Eager Bean

Lazy Bean → Eager Dependency

The eager dependency is usually created during startup.

The lazy bean is created only when it is needed.

When the lazy bean is finally created, Spring injects the already created dependency into it.

42. Lazy Bean Depends on Lazy Bean

Lazy Bean → Lazy Dependency

Both beans are not created at startup.

When the first lazy bean is requested, Spring creates it.

If it directly needs the second lazy bean, Spring may create the second bean at that time as well.

If `@Lazy` is used on the injection point, Spring may inject a proxy and delay the actual dependency creation until a method is called.

Part 17: How `@Lazy` Can Help in Circular Dependency

43. Using `@Lazy` to Break Constructor Circular Dependency

Sometimes `@Lazy` can help break a circular dependency by injecting a proxy instead of the real object immediately.

Example:

```
@Service
public class OrderService {
```

```

private final PaymentService paymentService;

public OrderService(@Lazy PaymentService paymentService) {
    this.paymentService = paymentService;
}
}

```

```

@Service
public class PaymentService {

    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}

```

Here, Spring does not need to create the real `PaymentService` immediately while creating `OrderService`.

It can inject a proxy.

The real `PaymentService` is resolved later when it is actually used.

44. Important Warning

Using `@Lazy` to solve circular dependency should be treated as a workaround, not the best design.

It may help in some situations, but the better solution is still refactoring.

Use `@Lazy` only when:

```

You understand the dependency flow clearly
Refactoring is not immediately possible
You are dealing with legacy code
You want a temporary solution

```

The best long-term solution is:

```

Fix the design
Separate responsibilities

```


Part 18: Bean Lifecycle

45. What Is Bean Lifecycle?

Bean lifecycle means:

The complete journey of a Spring bean from the moment Spring discovers it, creates it, injects dependencies into it, initializes it, uses it, and finally destroys it.

At a high level, the lifecycle looks like this:

```
Bean Definition
  ↓
Object Creation
  ↓
Dependency Injection
  ↓
Initialization
  ↓
Ready to Use
  ↓
Destruction
```

46. Complete Journey of a Spring Bean

Step 1: Bean Definition Is Created

Spring first discovers the bean.

This can happen through:

```
@Component
@Service
@Repository
@Controller
```

```
@Bean method  
XML configuration
```

At this stage, Spring has not necessarily created the actual object. It has only stored the bean definition.

The bean definition contains information like:

```
bean name  
class name  
scope  
dependencies  
lazy/eager behavior  
lifecycle methods
```

Step 2: Object Is Created

Spring creates the actual object using the constructor.

Example:

```
PaymentService paymentService = new PaymentService();
```

But in real Spring code, we do not write this manually.

The Spring container does it for us.

Step 3: Dependencies Are Injected

After object creation, Spring injects the required dependencies.

Example:

```
@Service  
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

```
}  
}
```

Spring provides `PaymentService` to `OrderService` .

Step 4: Bean Is Initialized

After dependency injection, Spring runs initialization logic.

This is the stage where we can perform setup tasks such as:

```
validating configuration  
opening resources  
loading required data  
checking required fields
```

Initialization can be done using different mechanisms such as:

```
@PostConstruct  
InitializingBean  
custom init method
```

Step 5: Bean Is Ready to Use

After initialization, the bean is ready.

Other beans can use it.

The application can now call its methods.

Step 6: Bean Is Destroyed

When the Spring container shuts down, singleton beans are destroyed.

This is where cleanup logic can run, such as:

```
closing resources  
releasing connections  
stopping background tasks
```

Destruction can be handled using:

```
@PreDestroy  
DisposableBean  
custom destroy method
```

For prototype beans, Spring creates the object and injects dependencies. Spring does not fully manage the destruction lifecycle in the same way as singleton beans.

Part 19: Final Summary

47. Circular Dependency Summary

Circular dependency happens when two or more beans depend on each other.

Example:

```
A needs B  
B needs A
```

Constructor-based circular dependency usually fails because constructor injection requires all dependencies before object creation.

Setter and field injection may sometimes work because object creation and dependency injection are separate steps.

But circular dependency is usually a design smell.

The best solution is not to depend on hacks, but to refactor the code.

48. Scope Summary

Scope	Meaning
singleton	One object per bean definition inside the Spring container
prototype	New object every time the bean is requested from the container

<code>request</code>	One object per HTTP request
<code>session</code>	One object per user session
<code>application</code>	One object per web application / ServletContext

Important point:

Spring singleton means one object per bean definition inside one object per class in the whole JVM.

49. Initialization Summary

Type	Meaning
Eager initialization	Bean is created during application startup
Lazy initialization	Bean is created only when needed

By default:

Singleton beans → eagerly initialized
 Prototype beans → created when requested

`@Lazy` can be used to delay bean creation.

`@Lazy` on an injection point creates a proxy and resolves the real dependency later.

Spring Boot also supports global lazy initialization:

```
spring.main.lazy-initialization=true
```

But lazy initialization can delay errors until runtime.

50. Key Takeaways

1. Spring creates beans based on dependency order.
2. Circular dependency means the dependency order becomes a loop.
3. Constructor-based circular dependency usually fails.

4. Setter or field injection may allow Spring to resolve some dependencies.
5. Spring Boot discourages circular references and disables
6. Circular dependency is usually a design smell and should refactoring.
7. Singleton scope means one object per bean definition in container.
8. Prototype scope means a new object is created whenever the bean is requested from the container.
9. Web scopes like request, session, and application are mainly used in web applications.
10. Eager initialization helps catch errors early during startup.
11. Lazy initialization delays bean creation until needed.
12. `@Lazy` on an injection point injects a proxy and resolves the real bean later.
13. Bean lifecycle covers the complete journey from bean definition to destruction